Table des matières

Cours 09 — Rebâtir le projet de bout en bout 1
Ce que vous allez faire 1
Avant de commencer 2
Étape 1 — La documentation sans trou · De bout en bout 2
Étape 2 — La commande qui rebâtit, et sa garde · De bout en bout 4
Étape 3 — Mettre de côté, puis effacer · De bout en bout 7
Étape 4 — Rebâtir depuis une copie neuve du dépôt · De bout en bout 8
Étape 5 — Prouver : les lignes, puis les témoins · De bout en bout 9
Étape 6 — Répondre au mandat du café · MARTS 11
Étape 7 — Passer le projet · De bout en bout 14
À retenir 16

Cours 09 — Rebâtir le projet de bout en bout

Cours 9/9 · La Brûlerie — Formation Analytics Engineering

STAGING	INTERMEDIATE	MARTS
reprendre la `RAW`, une vue par table	joindre, clarifier, préparer l'étoile	concevoir et livrer les étoiles
✓ ici · De bout en bout, le projet dbt complet, assemblé	✓	✓

Ce que vous allez faire

Huit cours ont bâti le projet morceau par morceau, sur un entrepôt qu'on n'a jamais effacé : ses tests passent, mais rien ne dit encore qu'il se refait. Un projet est fini quand il se rebâtit de zéro depuis son dépôt, qu'une preuve dit qu'il est revenu le même, et qu'une autre personne peut le reprendre sans vous. Ce cours n'apprend presque rien de neuf : il met le projet à ces trois épreuves. Il efface tout ce que vous avez bâti depuis le cours 03, le rebâtit d'une commande depuis une copie neuve du dépôt, le prouve, répond au mandat du café, puis étiquette la version livrée.

© 2026 iAlexMG. Tous droits réservés.

la documentation sans trou → la commande qui rebâtit, et sa garde → mettre de côté, puis effacer → rebâtir depuis une copie neuve → prouver : les lignes et les témoins → répondre au mandat du café → passer le projet

Les fichiers à écrire se donnent en pseudocode, et le code complet est dans `Codes_09_AE.md`, à ouvrir après avoir essayé.

Avant de commencer

Deux fenêtres de l'Anaconda Prompt servent à ce cours, toutes deux placées dans `Projet_Brulerie` : la première pour le Bloc-notes, dbt, DuckDB, Git et les scripts ; la seconde pour le notebook, qui s'ouvre à l'étape 6. Dans la première :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
set DBT_PROJECT_DIR=02_ae
REM Attendu : rien ne s'affiche.
dbt build
REM Attendu : 43 of 43, puis Done. PASS=43 WARN=0 ERROR=0.
git -C 02_ae status
REM Attendu : On branch main, ou master selon votre Git, puis nothing to commit, working tree clean.
```

Ce cours part du projet tel que le cours 08 l'a laissé, commité, sur la branche principale. Un fichier modifié et pas commité ferait mentir tout le cours : la reconstruction part d'une copie du dépôt, qui ne voit que ce que Git suit. S'il en reste un, reprenez la fin du cours 08 avant de continuer.

Étape 1 — La documentation sans trou · De bout en bout

Tout ce que ce cours ajoute au dépôt passe par une branche, comme n'importe quel changement proposé depuis le cours 08 : la CI la jugera à l'étape 7, avant qu'elle rejoigne la branche principale.

La documentation du cours 05 décrit les dimensions et les faits, ceux qu'on lit. Les vues du `STAGING`, `int_ventes` et le snapshot n'y ont que leur nom. Une description dit le grain d'un modèle et ce qu'il a déjà fait à la source, en une phrase, pas son SQL : c'est elle que lit la personne qui cherche une table, et le programme qui lit le manifeste. Le manifeste, `02_ae\target\manifest.json`, porte tout ce que dbt sait du projet (cours 05, étape 5) : il range chaque modèle et le snapshot sous `nodes`, avec leur sorte, `resource_type`, et leur `description`, vide quand le projet n'en écrit pas.

1.1 Créer la branche, dans la première fenêtre. `switch -c` crée une branche et s'y place.

```
git -C 02_ae switch -c bout-en-bout
REM Attendu : Switched to a new branch 'bout-en-bout'.
```

1.2 Chercher ce qui manque. `dbt parse` lit le projet et écrit le manifeste, sans toucher l'entrepôt ; une ligne de Python y prend le nom de chaque modèle et du snapshot dont la description est vide.

```
dbt parse
REM Attendu : Performance info: 02_ae\target\perf_info.json.
```


© 2026 iAlexMG. Tous droits réservés.

```
python -c "import json; m = json.load(open('02_ae/target/manifest.json', encoding='utf-8')); print(sorted(n['name'] for n in
m['nodes'].values() if n['resource_type'] in ('model', 'snapshot') and not n['description']))"
REM Attendu : ['int_ventes', 'snp_prix', 'stg_consentements', 'stg_lignes', 'stg_lots',
REM 'stg_membres', 'stg_produits', 'stg_tickets'].
```

Huit objets, dont la documentation ne sait que le nom. Les sources, elles, ont reçu leur description le jour où on les a déclarées.

1.3 Décrire les six vues du `STAGING`.

```
notepad 02_ae\models\staging\_staging.yml
```

Un fichier YAML de plus dans le dossier : dbt les lit tous, comme `_sources.yml`. Accepter la création, puis écrire une description par vue, dans vos mots :

```
# 02_ae/models/staging/_staging.yml : les six vues du STAGING, chacune avec son grain et ce qu'elle fait à la RAW
models:
  # 1. la caisse : de chaque ticket et de chaque ligne, le dernier export, donc le 12 juin une seule fois
  - name: stg_tickets
    ____: "Un ticket par ligne, typé, tiré de son export le plus récent."
  # puis, à écrire : stg_lignes, son grain, son export le plus récent et l'instant de cet export
  # 2. le catalogue et les membres : un seul export, typé et décodé
  # puis, à écrire : stg_produits et stg_membres ; celle des membres dit ce qui n'y monte pas
  # 3. les deux journaux, typés : un lot, une décision par ligne
  # puis, à écrire : stg_lots et stg_consentements
```

Enregistrer, puis fermer le Bloc-notes.

1.4 Décrire `int_ventes`, puis le snapshot. Le snapshot se décrit dans son propre dossier, que dbt lit comme `models`, sous la clé `snapshots:`.

```
notepad 02_ae\models\intermediate\_intermediate.yml
```

Accepter la création, puis écrire :

```
# 02_ae/models/intermediate/_intermediate.yml : la vue du milieu, avec la règle qu'elle porte
models:
  # 1. ce qui est une vente se décide ici, une fois, pour tout ce qui lit les ventes
  - name: ____
    description: "Les lignes des tickets payés, avec leur date, leur membre et l'instant de leur export : les annulés sortent, le
remboursement reste."
```

Enregistrer, fermer le Bloc-notes, puis :

```
notepad 02_ae\snapshots\_snapshots.yml
```

Accepter la création, puis écrire :

© 2026 iAlexMG. Tous droits réservés.

```
# 02_ae/snapshots/_snapshots.yml : l'histoire des prix, et ce qu'il faut savoir avant de l'effacer
____:
  # 1. la seule table qui porte une histoire : elle se rebâtit parce que le journal du catalogue la garde
  - name: snp_prix
    description: "Une version par prix de chaque produit, tirée du journal du catalogue depuis 2022 ; elle se rebâtit regard par
regard, du plus ancien au plus récent."
```

Enregistrer, puis fermer le Bloc-notes.

1.5 Chercher de nouveau.

```
dbt parse
python -c "import json; m = json.load(open('02_ae/target/manifest.json', encoding='utf-8')); print(sorted(n['name'] for n in
m['nodes'].values() if n['resource_type'] in ('model', 'snapshot') and not n['description']))"
REM Attendu : [], une liste vide.
```

Étape 2 — La commande qui rebâtit, et sa garde · De bout en bout

Rebâtir le projet demande plus qu'un `dbt build`. L'histoire des prix d'abord : `dbt build` prend le snapshot à son jour de départ, le 30 juin, et des regards plus anciens, pris ensuite, ouvriraient des versions datées d'avant celles qu'ils ferment (cours 05, étape 3). Les quatre regards se prennent donc dans l'ordre, puis `dbt build` bâtit le reste et le teste, et `dbt docs generate` refait la documentation, qui lit ce qui est bâti. Le fait des ventes s'y bâtit d'un bloc : sa table n'existe pas encore, et les passes incrémentales reprendront à la suivante.

Sept commandes, dont quatre ne diffèrent que par une date : personne ne les retape sans faute un jour de panne. Elles s'écrivent une fois, dans un script à la racine du dépôt, et chacune se termine par `|| exit /b 1`. `||` lance ce qui le suit quand la commande a échoué, et `exit /b 1` arrête le script avec le code 1 : sans lui, le script continuerait après un échec, et sa dernière ligne annoncerait un vert que rien ne justifie.

Lancé sur un entrepôt qui porte déjà le projet, ce script rejouerait l'histoire des prix par-dessus la sienne. Mesuré : le snapshot passe de 70 versions à 135, dont 27 qui finissent avant de commencer ; le 15 juin 2024, 21 produits ont deux prix en vigueur, le Latte deux fois 5,50 $. Et `dbt build` rend `PASS=43` : le prix d'époque ne regarde que juin 2025, où chaque produit n'a toujours qu'une version. Avant d'écrire la commande, on écrit donc ce qui l'empêche de tourner au mauvais endroit : une garde, qui compte ce que le projet a bâti et refuse s'il y en a un seul objet. Elle ne fait que regarder. Un script qui effacerait lui-même ce qu'il va rebâtir effacerait un jour le mauvais entrepôt ; un script qui refuse oblige quelqu'un à décider.

La garde est une macro : un morceau de Jinja qui porte un nom, et que dbt lance seul par `dbt run-operation`.

Pour écrire la garde	**L'outil**
nommer le morceau de Jinja	`{% macro exiger_entrepot_neuf() %}` au début, `{% endmacro %}` à la fin

© 2026 iAlexMG. Tous droits réservés.

Pour écrire la garde	L'outil
ce que l'entrepôt porte	`information_schema.tables`, une ligne par table et par vue, avec son entrepôt, `table_catalog` ; `current_database()` rend celui où dbt est connecté
les seuls objets du projet	`regexp_matches(table_name, '…')`, vrai pour un nom qui commence par l'un des préfixes d'Analytics Engineering, `stg_`, `int_`, `dim_`, `fct_` ou `snp_` : l'entrepôt est partagé, et `journal`, `raw_courbes` ou `argent_lots` sont à Data Engineering
garder le texte d'une requête	`{% set requete %}` … `{% endset %}` : le bloc entier devient la variable
la lancer, puis lire sa réponse	`run_query(requete)` rend un tableau ; `.columns[0].values()[0]`, sa première valeur
arrêter dbt sur un message	`exceptions.raise_compiler_error("…")`, qui rend le code 1 ; ~ colle deux textes
écrire au journal de dbt	`log("…", info=True)`

2.1 Écrire la garde.

```
mkdir 02_ae\macros
notepad 02_ae\macros\exiger_entrepot_neuf.sql
```

Le fichier est du Jinja pur : ses commentaires s'écrivent `{# … #}`, comme ceux du snapshot. Accepter la création, puis écrire :

```
{# 02_ae/macros/exiger_entrepot_neuf.sql : la garde, qui refuse de rebâtir sur un entrepôt déjà bâti #}
{% macro ____() %}
    {# 1. ce que le projet a bâti, à ses préfixes ; Data Engineering a aussi ses tables ici #}
    {% set requete %}
        select count(*) from ____
        where table_catalog = current_database()
            and regexp_matches(table_name, '^(stg|int|dim|fct|snp)_')
    {% endset %}
    {% set objets = run_query(requete).columns[0].values()[0] %}
    {# 2. un seul suffit : l'histoire des prix, rejouée par-dessus la sienne, s'abîmerait #}
    {% if objets > 0 %}
        {{ exceptions.____(
            "L'entrepôt porte déjà " ~ objets ~ " objets du projet : rien n'a été touché."
        ) }}
    {% endif %}
```


© 2026 iAlexMG. Tous droits réservés.

```
    {{ log("Entrepôt neuf : il n'a que la RAW.", info=True) }}
{% endmacro %}
```

Enregistrer, puis fermer le Bloc-notes.

2.2 Essayer la garde sur votre entrepôt, qui porte le projet entier.

```
dbt run-operation exiger_entrepot_neuf
REM Attendu : Encountered an error while running operation, puis L'entrepôt porte déjà 14 objets
REM du projet : rien n'a été touché.
echo %ERRORLEVEL%
REM Attendu : 1.
```

Quatorze objets : les six vues du `STAGING`, `int_ventes`, les trois dimensions, les trois faits et le snapshot. Rien n'a bougé : la garde n'a fait que compter.

2.3 Écrire la commande.

```
notepad 02_ae\reconstruire.cmd
```

Un fichier `.cmd` est une suite de commandes que cmd lance l'une après l'autre, comme si vous les tapiez. Il s'écrit sans accents : cmd le lit dans l'ancienne page de code de la console, et un accent y sortirait de travers.

Pour écrire le script	**L'outil**
ne pas afficher les commandes elles-mêmes, seulement ce qu'elles écrivent	`@echo off`, en première ligne
un commentaire	une ligne qui commence par `rem`
garder pour le script ce qu'il règle	`setlocal` : une variable posée ensuite s'oublie à la fin du script, et votre fenêtre garde la sienne
le dossier du script, où que soit la copie	`%~dp0`, qui finit par une barre inverse

Accepter la création, puis écrire :

```
@echo off
rem 02_ae/reconstruire.cmd : rebatir le projet de zero, en une commande, sur un entrepot qui n'a que la RAW
setlocal
rem 1. le projet est le dossier de ce script, ou que soit la copie ; l'entrepot est celui du profil
set DBT_PROJECT_DIR=____
rem 2. la garde d'abord : rien ne s'ecrit sur un entrepot qui porte deja le projet
dbt run-operation ____ || exit /b 1
rem 3. l'histoire des prix, regard par regard, du plus ancien au plus recent
dbt snapshot --vars "{jour: 2022-12-31}" || exit /b 1
```


© 2026 iAlexMG. Tous droits réservés.

```
rem puis les trois autres regards du cours 05, dans l'ordre, chacun avec son || exit /b 1
rem 4. tout le reste, tests compris, puis la documentation, qui lit ce qui est bati
____ || exit /b 1
dbt docs generate --static || exit /b 1
echo Reconstruction : au vert.
```

Enregistrer, puis fermer le Bloc-notes.

2.4 Lancer la commande sur votre entrepôt.

```
02_ae\reconstruire.cmd
REM Attendu : l'erreur de la garde, L'entrepôt porte déjà 14 objets du projet : rien n'a été touché.
echo %ERRORLEVEL%
REM Attendu : 1.
```

Le script s'est arrêté à sa première commande, et aucun regard n'a été pris : la garde est branchée. C'est la dernière fois qu'il tourne sur un entrepôt bâti.

2.5 Versionner ce que les étapes 1 et 2 ont écrit : la copie neuve de l'étape 4 ne verra que cela.

```
git -C 02_ae add models snapshots macros reconstruire.cmd
git -C 02_ae commit -m "Cours 09 : la documentation, la garde et la reconstruction"
REM Attendu : 5 files changed.
```

Étape 3 — Mettre de côté, puis effacer · De bout en bout

La première preuve comparera l'entrepôt rebâti à celui d'avant, et celui-ci n'existera plus une fois effacé : on en garde d'abord une copie. Puis on efface ce qu'Analytics Engineering a bâti, et rien d'autre. « De zéro » ne veut pas dire « de rien » : la `RAW` reste, puisqu'elle appartient à Data Engineering, et votre profil aussi, hors du dépôt.

3.1 Mettre de côté, dans la première fenêtre. Une copie de fichier, pas une exportation : près de 6 Mo, en un instant.

```
copy plateforme\entrepot\brulerie.duckdb plateforme\entrepot\brulerie_avant.duckdb
REM Attendu : 1 fichier(s) copié(s).
```

3.2 Écrire l'effacement. Le geste est celui de l'étape 1.2 du cours 08 : une requête écrit les instructions, que DuckDB lancera ensuite. Dans `information_schema.tables`, chaque objet a son type, `BASE TABLE` ou `VIEW` ; la requête fait, de chaque objet du projet, reconnu à son préfixe comme par la garde, son `drop`. Les tables de Data Engineering restent : chaque formation n'efface que les siennes.

```
duckdb -readonly plateforme\entrepot\brulerie.duckdb -noheader -list -c "select 'drop ' || case when table_type = 'VIEW' then
'view' else 'table' end || ' ' || table_name || ';' from information_schema.tables where regexp_matches(table_name,
'^(stg|int|dim|fct|snp)_') order by table_name" > effacer.sql
type effacer.sql
REM Attendu : quatorze lignes, de drop table dim_date; à drop view stg_tickets;
```


© 2026 iAlexMG. Tous droits réservés.

Le fichier vit dans `Projet_Brulerie`, hors du dépôt : il ne sert qu'une fois.

3.3 Effacer, puis regarder ce qui reste.

```
duckdb plateforme\entrepot\brulerie.duckdb -f effacer.sql
REM Attendu : rien ne s'affiche, l'invite revient.
duckdb -readonly plateforme\entrepot\brulerie.duckdb -c "show tables"
REM Attendu : neuf tables, journal et les huit raw_…, de raw_consentements à raw_tickets, plus
REM argent_lots et direct_courbes si vous avez fait Data Engineering.
```

Il reste la `RAW`, que vous n'avez jamais écrite. L'histoire des prix est partie avec le reste ; elle se rebâtira parce que la `RAW` garde le journal du catalogue, ses 118 modifications depuis 2022. Si la source ne gardait que son état courant, comme la plupart des systèmes, effacer le snapshot perdrait cette histoire pour de bon : il se sauvegarderait, et ce serait la seule table du projet à l'exiger.

Étape 4 — Rebâtir depuis une copie neuve du dépôt · De bout en bout

La reconstruction ne part pas de votre dossier de travail, qui porte `target`, des journaux, et peut-être un fichier oublié. Elle part de ce que Git suit, comme le ferait un collègue, et comme la CI du cours 08. La copie se fait à côté du dépôt, dans `Projet_Brulerie` ; votre profil reste où il est et sert tel quel : c'est le même entrepôt.

4.1 Copier le dépôt. Un clone prend la branche où se trouve le dépôt qu'il copie.

```
git clone --quiet 02_ae 02_ae_neuf
git -C 02_ae_neuf branch --show-current
REM Attendu : bout-en-bout.
dir /b 02_ae_neuf
REM Attendu : .gitignore, .sqlfluff, ci, dbt_project.yml, macros, models, reconstruire.cmd, snapshots
REM et tests ; ni target ni logs.
```

4.2 Rebâtir. Comptez une petite minute.

```
02_ae_neuf\reconstruire.cmd
REM Attendu : Entrepôt neuf : il n'a que la RAW. ; quatre fois 1 of 1 OK snapshotted main.snp_prix ;
REM Done. PASS=43 ; Catalog written to …\02_ae_neuf\target\catalog.json ; puis Reconstruction : au vert.
echo %DBT_PROJECT_DIR%
REM Attendu : 02_ae.
```

dbt prévient d'abord qu'il relit tout le projet : la copie n'a pas encore de `target`. Il le redit à chaque regard, dont les variables changent. Le temps se répartit mal, et c'est instructif : mesuré, une cinquantaine de secondes en tout, dont deux pour bâtir et tester les 43 nœuds. Le reste est le démarrage de dbt, qui relit le projet à chacune des sept commandes : une reconstruction se compte en commandes, pas en lignes. Et `setlocal` a tenu parole : votre fenêtre travaille toujours sur `02_ae`.

© 2026 iAlexMG. Tous droits réservés.

4.3 Regarder l'histoire des prix rebâtie. `count(dbt_valid_to)` ne compte que les versions qui ont une fin, et un `case` sans `else` rend vide ce qu'il ne retient pas.

```
duckdb -readonly plateforme\entrepot\brulerie.duckdb -c "select count(*) as versions, count(dbt_valid_to) as fermees, count(case when dbt_valid_to < dbt_valid_from then 1 end) as a_l_envers from snp_prix"
REM Attendu : 70 versions, 38 fermées, 0 à l'envers, comme au cours 05.
```

Étape 5 — Prouver : les lignes, puis les témoins · De bout en bout

`Reconstruction : au vert` dit que chaque commande a rendu 0, donc que chaque test est passé ; l'étape 2 a montré ce que vaut ce vert sur une histoire abîmée. Refaire n'est pas prouver. Trois preuves s'empilent, chacune voyant ce que les autres ne voient pas ; la troisième est l'étape 6.

La preuve	Ce qu'elle compare	Ce qu'elle ne voit pas
les lignes	chaque table rebâtie contre celle d'avant, dans les deux sens ; chaque vue par sa définition	ce qui était déjà faux avant
les témoins	l'étoile contre un chemin qui ne passe pas par elle : la caisse, le journal du catalogue, la fiche des membres	ce qu'aucun témoin ne couvre
le mandat du café	l'étoile contre les questions pour lesquelles elle existe	une réponse juste pour de mauvaises raisons

Le script de la première attache les deux entrepôts côte à côte, en lecture seule, et prend de chaque côté ce que le projet bâtit. Une table se compare par ses lignes, dans les deux sens, par `except all` : les lignes de la première requête absentes de la seconde, exemplaire par exemplaire, si bien qu'une ligne en double d'un seul côté se voit. L'opérateur s'apprend au Cours 07 de la formation SQL avancé, `Theorie_07_SQL.md`, section 6.

Une vue, elle, se compare par sa définition, et le piège vaut d'être connu. DuckDB range dans chaque vue le nom de l'entrepôt qu'elle lit : `stg_tickets` lit `brulerie.main.raw_tickets`. Ouverte à côté de l'entrepôt rebâti, une vue de la copie d'avant lit donc les tables du rebâti, et ses lignes ne prouveraient rien. Son contenu suit de sa définition et des tables, qui sont comparées.

Pour écrire la comparaison	L'outil
les deux chemins donnés après le nom du script	`sys.argv[1]` et `sys.argv[2]`
la définition d'une vue	la colonne `sql` de `duckdb_views()`, une ligne par vue de chaque entrepôt attaché ; deux définitions pareilles n'y font qu'une valeur distincte
les noms d'un seul côté, puis ceux des deux	`a.keys() ^ b.keys()` et `a.keys() & b.keys()`, sur les deux dictionnaires
le code de sortie du script	`sys.exit(1)` en cas d'écart, comme la CI du cours 08

© 2026 iAlexMG. Tous droits réservés.

5.1 Écrire la comparaison.

```
mkdir 02_ae\outils
notepad 02_ae\outils\comparer.py
```

Accepter la création, puis écrire :

```
# 02_ae/outils/comparer.py : l'entrepôt rebâti contre celui d'avant, objet par objet
import sys

import duckdb

# 1. les deux entrepôts, attachés côte à côte en lecture seule : celui d'avant, puis le rebâti
con = duckdb.connect()
con.execute(f"attach '{sys.argv[1]}' as avant (____)")
# puis, à écrire : le second chemin, attaché de même sous le nom rebati
# 2. les objets du projet, à leurs préfixes, avec leur type ; un objet d'un seul côté est un écart
objets = ("select table_name, table_type from ____ where table_catalog = ? "
          "and regexp_matches(table_name, '^(stg|int|dim|fct|snp)_')")
avant, rebati = (dict(con.execute(objets, [base]).fetchall()) for base in ("avant", "rebati"))
ecarts = len(avant.keys() ^ rebati.keys())
for nom in sorted(avant.keys() & rebati.keys()):
    if rebati[nom] == "VIEW":
        # 3. une vue, par sa définition, la même des deux côtés : elle nomme l'entrepôt qu'elle lit
        vue = "select count(distinct sql) - 1 from ____ where view_name = ?"
        # ecart : la réponse de vue pour ce nom ; etat : « vue, définition identique » ou différente
    else:
        # 4. une table, par ses lignes, dans les deux sens : ce qui manque, puis ce qui est en trop
        sens = "select count(*) from (select * from {} ____ select * from {})"
        # manquent : avant contre rebati ; en_trop : rebati contre avant ; ecart : leur somme
    ecarts += ecart
    print(f"{nom:<18} {etat}")
print(f"{len(rebati)} objets rebâtis, {len(avant)} avant : {ecarts} écart(s)")
sys.exit(1 if ecarts else 0)
```

Enregistrer, puis fermer le Bloc-notes.

5.2 Comparer.

```
python 02_ae\outils\comparer.py plateforme\entrepot\brulerie_avant.duckdb plateforme\entrepot\brulerie.duckdb
REM Attendu : quatorze lignes, de dim_date à stg_tickets, puis 14 objets rebâtis, 14 avant : 0 écart(s).
echo %ERRORLEVEL%
REM Attendu : 0.
```


© 2026 iAlexMG. Tous droits réservés.

Le début de la sortie, et sa fin :

```
dim_date           table, 0 ligne(s) manquent, 0 en trop
dim_membre         table, 0 ligne(s) manquent, 0 en trop
dim_produit        table, 0 ligne(s) manquent, 0 en trop
fct_consentements  table, 0 ligne(s) manquent, 0 en trop
fct_lots           table, 0 ligne(s) manquent, 0 en trop
fct_ventes         table, 0 ligne(s) manquent, 0 en trop
int_ventes         vue, définition identique
snp_prix           table, 0 ligne(s) manquent, 0 en trop
...
14 objets rebâtis, 14 avant : 0 écart(s)
```

Sept tables égales ligne pour ligne, sept vues à la définition identique. `fct_ventes`, bâti par passes au cours 07 et d'un bloc ici, porte les mêmes lignes : un fait incrémental bien filtré égale le même fait bâti d'un coup. L'histoire des prix, rejouée de zéro, revient aussi à l'identique, jusqu'à l'identifiant de chaque version. Mesuré sur une copie abîmée exprès (une ligne du fait retirée, un ticket en double, une vue changée, une table que rien ne bâtit plus) : 5 écarts, et le code 1.

5.3 Interroger les témoins. La comparaison dit que rien n'a bougé ; elle ne peut pas dire que c'était juste. Les tests singuliers des cours 05 et 06 opposent l'étoile à autre chose qu'elle-même, et `test_type:singular` ne choisit qu'eux. Ils ont tourné dans la reconstruction ; on les relance seuls pour les lire.

```
dbt test --select test_type:singular
REM Attendu : 4 of 4, puis Done. PASS=4 WARN=0 ERROR=0.
```

Le test	Ce qu'il oppose
`assert_grain_fct_ventes`	le fait à son grain : aucun couple ticket et ligne deux fois
`assert_temoin_caisse`	chaque ticket payé du fait au sous-total que la caisse a imprimé
`assert_prix_d_epoque`	chaque ligne vendue au prix que le journal du catalogue donnait ce jour-là
`assert_temoin_consentements`	la dernière décision de chaque membre à sa fiche

Leur limite : aucun ne regarde l'histoire avant juin 2025, et l'étape 2 l'a mesuré, 27 versions à l'envers sous un `PASS=43`. Contre ce défaut-là, seule la comparaison protège, à condition d'avoir mis l'entrepôt d'avant de côté.

Étape 6 — Répondre au mandat du café · MARTS

La dernière preuve ne compare rien à rien : l'étoile rebâtie répond-elle encore à ce que le café a demandé ? Trois questions, celles de la mise en situation, puis une quatrième, qui croise deux faits.

6.1 Ouvrir le notebook. Seconde Anaconda Prompt : activer `formations`, se placer dans `Projet_Brulerie`, puis lancer DuckDB.

© 2026 iAlexMG. Tous droits réservés.

```
duckdb -ui
```

Créer un notebook et le renommer « AE Cours 09 ». Dans la première cellule, attacher l'entrepôt :

```
attach if not exists 'plateforme/entrepot/brulerie.duckdb' as brulerie (read_only);
-- Attendu : brulerie paraît sous Attached databases.
```

Chaque requête qui suit va dans une cellule neuve.

6.2 Qu'est-ce qui s'est vendu, par catégorie, puis par produit ?

```
-- 1. le montant vendu par catégorie, du plus grand au plus petit
select p.categorie, sum(f.quantite) as quantite, sum(f.montant) as montant
from brulerie.fct_ventes as f
inner join brulerie.dim_produit as p on p.cle_produit = f.cle_produit
group by p.categorie
order by montant desc;
-- Attendu : BOISSON_CHAUDE 1214 et 5638.00 ; GRAINS 173 et 3721.50 ; VIENNOISERIE 638 et 2407.25 ;
-- BOISSON_FROIDE 404 et 2291.25 ; SANDWICH 214 et 2033.00 ; ACCESSOIRE 38 et 627.00.

-- 1. les cinq produits qui ont rapporté le plus
select p.nom, sum(f.quantite) as quantite, sum(f.montant) as montant
from brulerie.fct_ventes as f
inner join brulerie.dim_produit as p on p.cle_produit = f.cle_produit
group by p.nom
order by montant desc
limit 5;
-- Attendu : Latte 264 et 1495.00 ; Latte glacé 227 et 1362.00 ; Cappuccino 212 et 1092.00 ;
-- Sandwich jambon et gruyère 114 et 1083.00 ; Sandwich végé 100 et 950.00.
```

6.3 Combien les membres ont-ils acheté, et combien les clients sans carte ? Un ticket sans carte a sa clé de membre vide, depuis le cours 04.

```
-- 1. les ventes avec et sans carte de membre : les tickets, et leur montant
select f.cle_membre is not null as avec_carte, count(distinct f.ticket_id) as tickets,
       sum(f.montant) as montant
from brulerie.fct_ventes as f
group by avec_carte
order by avec_carte;
-- Attendu : false, 973 tickets et 11111.00 ; true, 539 et 5607.00.
```

Un peu plus du tiers des tickets porte une carte, pour le tiers du montant. Les 1 512 tickets sont les 1 511 ventes payées et le remboursement, qui reste une vente au sens du cours 04.

6.4 Ces chiffres retombent-ils sur ceux de la caisse, au cent près ?

© 2026 iAlexMG. Tous droits réservés.

```sql
-- 1. le fait contre la caisse : la somme des lignes vendues, et celle des sous-totaux des tickets payés
select (select sum(montant) from brulerie.fct_ventes) as fait,
       (select sum(sous_total) from brulerie.stg_tickets where statut = 'PAYE') as caisse;
-- Attendu : 16718.00 des deux côtés.
```

Le chiffre du cours 04 a traversé une reconstruction de zéro sans perdre un cent.

6.5 Le café torréfié et le café vendu, côte à côte. Le cours 06 l'a promis : `dim_produit`, conforme, sert le fait des lots et celui des ventes. Deux faits ne se joignent jamais l'un à l'autre. Chacun s'additionne d'abord à son propre grain, puis les deux résultats se rangent côte à côte par la dimension qu'ils partagent. Joints directement, chaque lot se répéterait pour chaque ligne vendue du même café : pour le Mélange du Passage, 16 lots fois 39 lignes, 624 paires, et 1 296,75 kg torréfiés au lieu de 33,25. Un sac pèse 340 g, le format que donne le catalogue.

```sql
-- 1. chaque fait additionné à son grain, puis les deux rangés côte à côte par le produit
with torrefie as (
    select cle_produit, count(*) as lots, sum(poids_torrefie_kg) as kg_torrefies
    from brulerie.fct_lots
    group by ____
),
vendu as (
    select cle_produit, sum(quantite) as sacs
    from brulerie.fct_ventes
    group by cle_produit
)
select p.nom, t.lots, t.kg_torrefies, v.sacs, v.sacs * 0.340 as kg_vendus
from brulerie.dim_produit as p
inner join torrefie as t on t.cle_produit = p.cle_produit
left join vendu as v on v.cle_produit = p.cle_produit
order by t.kg_torrefies desc;
-- Attendu : huit cafés, du Mélange du Passage, 16 lots, 33.25 kg torréfiés, 41 sacs et 13.940 kg
-- vendus, au Brésil Sul de Minas, 2 lots, 2.53 kg, 14 sacs et 4.760 kg.
```

Pour quatre des huit cafés, juin a vendu en sacs plus qu'il n'a torréfié : le Guatemala, l'Éthiopie, le Déca et le Brésil. L'étoile ne dit rien du café torréfié avant le 1er juin, que la `RAW` ne porte pas ; ce que les faits ne portent pas, une requête ne l'invente pas. Personne n'a écrit cette table pour la reconstruction : elle sort de deux faits bâtis à deux cours différents, qui partagent une dimension. C'est la matrice de bus du cours 06, qui tient sur un entrepôt rebâti de zéro.

6.6 Fermer le notebook : la documentation de l'étape 7 ouvrira l'entrepôt. Fermer l'onglet du navigateur, puis quitter DuckDB dans la seconde Anaconda Prompt.

```
.quit
```

Attendu : l'invite revient, et la seconde fenêtre peut se fermer.

© 2026 iAlexMG. Tous droits réservés.

Étape 7 — Passer le projet · De bout en bout

Un projet se passe par écrit. La documentation, sans trou depuis l'étape 1, dit chaque modèle à qui cherche une table. Il manque ce que lit la personne qui reprendra le dépôt : par où commencer. C'est le `README.md`, à la racine du dépôt, que GitHub et les éditeurs affichent en premier. Il ne raconte pas le code, que la première ligne de chaque fichier explique : il dit quoi avoir avant de commencer, quoi lancer, d'où, et ce qui doit sortir. Son épreuve : quelqu'un qui ne vous connaît pas rebâtit le projet en le suivant, sans vous appeler.

7.1 Écrire le `README.md`, dans la première fenêtre.

```
notepad 02_ae\README.md
```

Un fichier Markdown : `#` ouvre un titre, `-` une puce, et l'accent grave entoure un nom de fichier ou une commande. `<!-- … -->` y est un commentaire, que l'affichage cache. Accepter la création, puis écrire chaque section dans vos mots :

```
# 02_ae — l'étoile des ventes de La Brûlerie du Passage

<!-- 1. ce qu'est le dépôt, et ce qu'il livre -->
(trois ou quatre phrases : le café, la RAW qu'il reprend, l'étoile et les deux faits qu'il en tire,
le chiffre qui prouve qu'il dit vrai)

## Ce qu'il faut avoir

<!-- 2. ce qui n'est pas dans le dépôt : l'entrepôt, le profil, les outils -->
- `Projet_Brulerie/plateforme/entrepot/brulerie.duckdb`, avec les sept tables de la `RAW` ;
(puis le profil, hors du dépôt, et l'environnement conda, avec les versions des outils)

## Rebâtir

<!-- 3. la commande, d'où la lancer, et ce qui doit sortir -->
(la commande de l'étape 2, ce qu'elle suppose, ce qu'elle affiche, et ce que fait la garde)

## Prouver et vérifier

<!-- 4. les preuves, puis la vérification d'un changement -->
(la comparaison, les tests singuliers, la CI du cours 08, chacun avec son attendu)
```

Enregistrer, puis fermer le Bloc-notes.

7.2 Versionner, puis passer la branche à la CI.

```
git -C 02_ae add outils README.md
git -C 02_ae commit -m "Cours 09 : la comparaison et le README"
REM Attendu : 2 files changed.
```


© 2026 iAlexMG. Tous droits réservés.

```
python 02_ae\ci\verifier.py
REM Attendu : quatre étapes, chacune annoncée par >, dont All Finished! et Done. PASS=43, puis
REM CI au vert : le dernier commit passe la forme et le fond, sur une copie neuve.
```

La CI a cloné la branche `bout-en-bout`. Elle la juge sur sa forme et sur ce qu'elle se bâtit ; le fond, ce sont les preuves des étapes 5 et 6 qui l'ont jugé, sur un entrepôt qui a des données. La garde, la documentation et le `README.md` y sont entrés sans rien casser.

7.3 Fusionner la branche, puis étiqueter la version. `merge --no-ff` garde la trace de la branche par un commit de fusion, même quand Git pourrait simplement avancer la branche principale ; `branch -d` efface une branche déjà fusionnée.

```
git -C 02_ae switch -
REM Attendu : Switched to branch 'main', ou 'master' selon votre Git.
git -C 02_ae merge --no-ff bout-en-bout -m "Cours 09 : le projet rebâti de zéro, prouvé et passé"
REM Attendu : Merge made by the 'ort' strategy., puis 7 files changed.
git -C 02_ae branch -d bout-en-bout
REM Attendu : Deleted branch bout-en-bout.
```

Reste à nommer l'état livré. Une étiquette annotée, `tag -a`, porte un auteur, une date et un message, comme un commit. `v1.0` nomme ce commit, et le nommera encore quand la branche principale aura avancé. Un changement qui casserait ceux qui lisent le fait, une colonne renommée comme au cours 08, ferait la `v2.0`.

```
git -C 02_ae tag -a v1.0 -m "La Brûlerie, rebâtie de zéro et prouvée"
git -C 02_ae log --oneline --decorate -1
REM Attendu : (HEAD -> main, tag: v1.0) Cours 09 : le projet rebâti de zéro, prouvé et passé ;
REM master à la place de main selon votre Git.
git -C 02_ae status
REM Attendu : nothing to commit, working tree clean.
```

Le dépôt `02_ae`, étiqueté `v1.0`, est la pièce que vous pouvez montrer.

7.4 Faire le ménage, puis rouvrir la documentation. La copie neuve, l'entrepôt d'avant et l'effacement ont servi ; rien de ce qui part n'est une donnée : la copie se refait par `git clone`, et l'entrepôt d'avant a fini son travail de témoin. La documentation de votre dossier de travail, elle, date d'avant l'étape 1 : la reconstruction a écrit la sienne dans la copie.

```
rmdir /s /q 02_ae_neuf
del plateforme\entrepot\brulerie_avant.duckdb effacer.sql
dir /b
REM Attendu : ni 02_ae_neuf, ni effacer.sql.
dir /b plateforme\entrepot
REM Attendu : brulerie.duckdb et ci.duckdb, l'entrepôt de la CI ; plus de brulerie_avant.duckdb.
dbt docs generate --static
REM Attendu : Building catalog, puis Catalog written to …\02_ae\target\catalog.json.
```


© 2026 iAlexMG. Tous droits réservés.

```
start 02_ae\target\static_index.html
REM Attendu : la documentation s'ouvre dans le navigateur.
```

Chercher `stg_tickets` dans la page : sa description dit maintenant son grain et ce qu'elle fait à la `RAW`, comme celle des faits.

À retenir

- Un projet est fini quand il se rebâtit de zéro depuis une copie neuve de son dépôt, par une commande qui s'arrête au premier échec, et qu'une garde empêche de tourner sur un entrepôt déjà bâti : rejouée par-dessus elle-même, l'histoire des prix s'abîme sans qu'un test le voie.
- Refaire n'est pas prouver : les tables se comparent ligne pour ligne dans les deux sens, les vues par leur définition, et les témoins opposent l'étoile à la caisse et aux journaux.
- Deux faits se croisent par leur dimension conforme, chacun additionné d'abord à son grain ; joints l'un à l'autre, ils se multiplient.

© 2026 iAlexMG. Tous droits réservés.